PROJE 17 / 20 · ÇALIŞAN TEMEL SÜRÜM

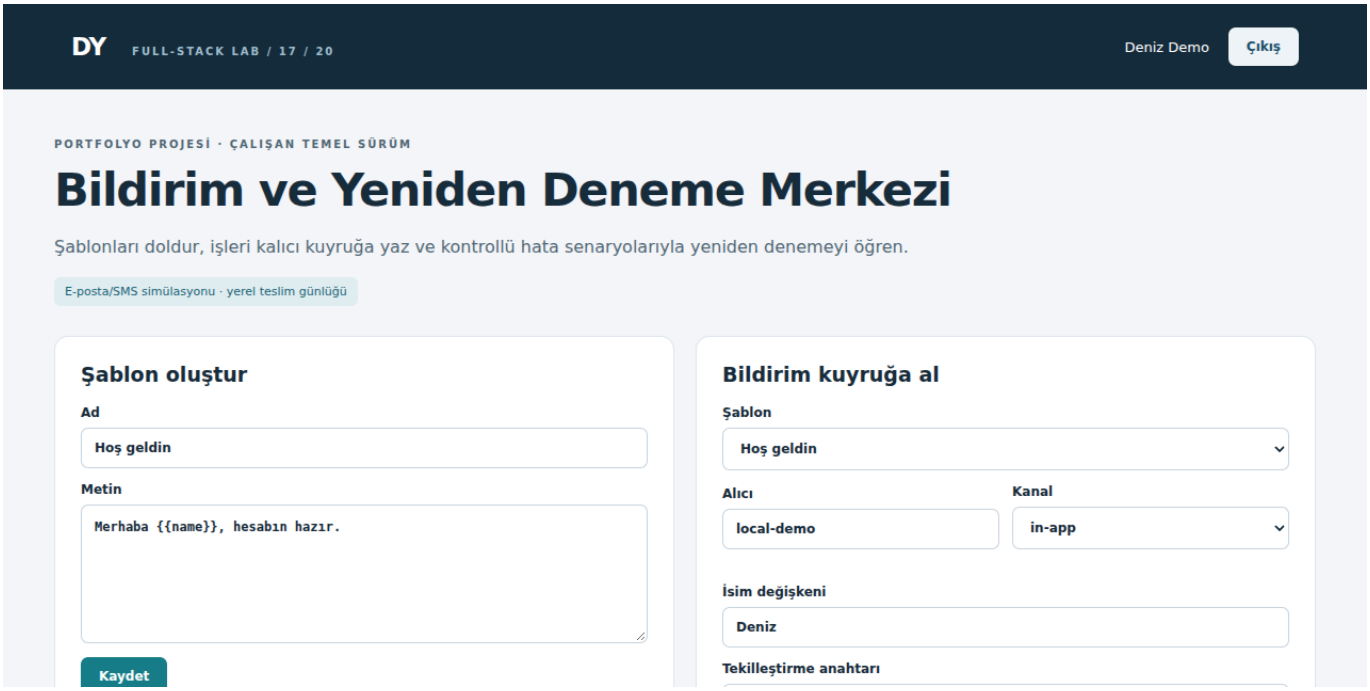
Bildirim ve Yeniden Deneme Merkezi

Şablonları doldur, işleri kalıcı kuyruğa yaz ve kontrollü hata senaryolarıyla yeniden denemeyi öğren.

E-posta/SMS simülasyonu · yerel teslim günlüğü

Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.



Gerçek uygulama ekranı · yerel demo verileri

Teslim edilen kapsam

İsim değişkenli şablon · Üç demo kanal · Dedupe anahtarı · Üstel yeniden deneme · Başarısız işi yeniden başlatma

Bu sürümde olmayanlar

Gerçek e-posta, SMS ve cihaz push gönderilmez. delivered durumu yerel simülatörün kabulüdür; gerçek alıcıya ulaştığı anlamına gelmez. Tek süreç worker kullanılır.

02 / Backend ve veri modeli

notifications (owner,dedupe) UNIQUE ile aynı işi tekilleştirir. Worker her saniye zamanı gelen işleri alır. retry-demo alıcısı üçüncü denemede başarılı olur; fail-demo üç denemede failed olur. Sonraki zaman 2^attempt saniyedir. Şablon eksik değişken içerirse iş kuyruğa girmeden 422 döner.

Gerçek SQL şeması

```
CREATE TABLE templates(id TEXT PRIMARY KEY,owner TEXT REFERENCES users(id),name TEXT,body
 TEXT);


 CREATE TABLE notifications(id TEXT PRIMARY KEY,owner TEXT REFERENCES users(id),template
TEXT REFERENCES templates(id),channel TEXT,recipient TEXT,body TEXT,status TEXT,attempt
INTEGER DEFAULT 0,next_at INTEGER,last_error TEXT,dedupe TEXT,UNIQUE(owner,dedupe));
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekilliği ve REFERENCES ilişkiyi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

Kod okuma sırası

Dosya	Ne arayacaksın?
client/Workspace.vue	Eylemi tetikleyen form ve API isteği
client/api.ts	JSON isteği, hata yakalama ve mesaj
server/project.js	Yetki kontrolü → kural → SQL → yanıt
server/core.js	Parametreli SQL, transaction, oturum, SSE

Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

Yöntem / yol	Girdi alanları	Başarı
GET /api/templates	Gövde yok	200
POST /api/templates	JSON: name, body	201
GET /api/notifications	Gövde yok	200
POST /api/notifications	JSON: templateId, dedupe, variables, channel, recipient	202
POST /api/notifications/:id/retry	Gövde yok	200

Somut istek örneği

```
POST /api/templates
{"name":"Welcome","body":"Merhaba {{name}}"}
201 {"id":"<template-id>"}
```

Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error":"açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

Kod	Anlam
401 / 403	Giriş gerekli / rol veya origin reddedildi
404 / 409	Kayıt görünmüyor / durum veya benzersizlik çakışması
422 / 429	Geçersiz girdi / giriş veya uygulama kotası
500 / 502 / 503	Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

- Eksik şablon değişkeni iş oluşturmadan 422 dönmeli.
- Aynı dedupe anahtarı aynı işi döndürmeli.
- Yerel başarılı alıcı delivered durumuna gelmeli.
- retry-demo için üçüncü deneme başarı, fail-demo için failed görülmeli.

Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

Test kodundan örnek

```
const t=await post('/templates',{name:'Welcome',body:'Hello {{name}}'},201);
await post('/notifications',{templateId:t.id,recipient:'local-demo',channel:'in-
app',dedupe:'missing',variables:{}},422);
const n=await post('/notifications',{templateId:t.id,recipient:'local-demo',channel:'in-
app',dedupe:'one',variables:{name:'Deniz'}},202);
const duplicate=await post('/notifications',{templateId:t.id,recipient:'local-
demo',channel:'in-app',dedupe:'one',variables:{name:'Deniz'}});assert.equal(n.id,duplica
te.id);assert.equal(duplicate.duplicate,true);
await new Promise(r=>setTimeout(r,1200));assert.equal((await
get('/notifications'))[0].status,'delivered');
```

Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

Doğrulamanın sınırı

API entegrasyon testleri, üretim derlemesi ve Vue tip/şablon kontrolü uygulandı. Bunlar yük testi, penetrasyon testi veya tüm tarayıcılarda kapsamlı uçtan uca test yerine geçmez.

05 / Çalışma ve GitHub planı

İlk günün somut hedefi

Şablon oluştur ve aynı dedupe ile iki gönderim yap. Tek satır oluştuğunu gör. retry-demo alıcısıyla artan deneme sayısını izle.

Çalışma bloğu	Teslim edeceğin çıktı
1 · Kurulum / 30-60 dk	Çalışan ekran, örnek veri ve ilk başarılı API isteği
2 · Kod okuma / 45-90 dk	Formdan SQL sorgusuna kadar bir isteğin açıklaması
3 · Geliştirme / 60-120 dk	Bir iş kuralı veya arayüz davranışında kendi değişikliğin
4 · Doğrulama / 30-60 dk	İlgili test ve README değişikliği; yeni ekran görüntüsü

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

İleri sürüm: dört gerçek geliştirme

- 1. Sağlayıcı adaptörleri ekle
- 2. Outbox ve ayrı worker kur
- 3. İş sahiplenme/lease mekanizması ekle
- 4. Webhook teslim raporlarını ayır

Anlamlı commit sırası

```
git add .
git commit -m "chore: add notification-hub baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

Mülakatta anlat

Teslim edildi, sağlayıcı kabul etti ve kullanıcı gördü durumları aynı mı? Bir iş iki worker tarafından alınırsa ne olur?

Özel kurulum / ilerleme notu

Önce yerel sürümü tamamla. Sonraki aşamada PostgreSQL, migration, pagination ve gerçek kullanıcı yönetimini ihtiyaca göre ekle. Ortak kurulum ayrıntıları README.md ve başlangıç rehberindedir.